

# DEAD AIR

*Pitch Capstone*

*Mi chiamo Michele Grimaldi e il mio Capstone Project è DEAD AIR*

---

*DEAD AIR*

**E' un gioco horror narrativo in Unity dove interpreti un operatore del 911 negli anni '90, rispondendo a chiamate di emergenza sempre più inquietanti.**

---

Prima di spiegare il progetto, voglio partire dal pattern architetturale che ne è il fondamento: l'**Observer Pattern**.

L'**Observer Pattern** risolve un problema classico nella programmazione: come fai a far **comunicare due sistemi senza che si conoscano direttamente**? La risposta è semplice -- metti un **intermediario in mezzo**. Hai un **Subject**, che genera eventi, e uno o più **Observer**, che li ascoltano e reagiscono. Il Subject non sa chi lo ascolta. Gli Observer non sanno da dove arriva il segnale. Tutto passa attraverso un canale condiviso.

In DEAD AIR ho implementato entrambi attraverso **14 Event Channels basati su ScriptableObject**. Ma non è sempre stato così -- inizialmente usavo un **hub statico** centralizzato. Il problema era chiaro: **un singolo punto di controllo crea accoppiamento**. Se l'hub cambia, tutto si rompe. Ho quindi **refactorizzato** l'architettura migrando agli **ScriptableObject**, dove ogni canale è un **asset Unity indipendente** -- un oggetto dati condiviso che fa da ponte tra i sistemi. Questo è l'Observer Pattern in azione. E poiché ogni ScriptableObject esiste come unica istanza condivisa in memoria tra tutti i sistemi che lo usano, questo è anche il **Flyweight in azione**. Ogni ScriptableObject esiste come **unica istanza in memoria**, condivisa da tutti i sistemi che ne hanno bisogno. **Nessuna duplicazione, nessuno spreco di memoria**.

Il flusso concreto è questo: **scrivo la storia in file Ink con tag speciali** come **#voice:iris\_01** o **#sfx:phone\_ring**. Il **DialogueParser** li legge e li trasforma in dati **type-safe** tramite **enum**.

---

Ovvero: invece di usare **stringhe libere per identificare i comandi** -- dove un errore di battitura passa inosservato a runtime -- usi un **enum**, che è un **tipo definito a compile-time**.

**Senza enum -- stringa libera:**

```
if (command == "dead_air_screen") // implica errori di battitura
```

**Con enum -- type-safe:**

```
if (command == UICommandType.DeadAirScreen) // se sbagli il nome, il compilatore si blocca subito
```

Lo StoryManager li pubblica sui canali. AudioManager, VoiceManager e DialogueUI -- ognuno ascolta solo il suo canale e reagisce.

In pratica: con le **stringhe** l'errore lo scopri **quando il gioco è in esecuzione** e qualcosa non funziona. Con gli **enum** l'errore lo scopri **prima ancora di avviare il gioco** -- il compilatore te lo segnala direttamente.

Questi dati lo **StoryManager** li pubblica sui canali. **AudioManager**, **VoiceManager** e **DialogueUI** -- ognuno ascolta solo il suo canale e reagisce.

**Risultato:** sistemi completamente **indipendenti**. **Puoi rimuoverne uno senza rompere gli altri**. Puoi **aggiungere una nuova storia in due minuti** senza toccare codice.

---

**DEAD AIR** è un **sistema architetturale scalabile** -- decoupling reale, event-driven, separazione delle responsabilità. Gli stessi principi del software professionale, applicati a un gioco.